

Reverse Engineering of Private Protocols: Methods, Tools, Challenges and Future Directions

Ke Shi¹, Xingchi Chen¹, Yuyu He^{1,*}, Yanli Chen^{1,2,3}, Zhigang Wang¹, Hui Lu^{1,4,5}

¹Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China

²Key Laboratory Ministry of Industry and Information Technology, Guangzhou, China

³Key Laboratory of Cyberspace Security, Ministry of Education, Zhengzhou, China

⁴Peng Cheng Laboratory, Shenzhen, China

⁵Quanzhou Normal University, Quanzhou, China

*Corresponding author: Yuyu He (heyu@gzhu.edu.cn)

Abstract—As the digital infrastructure grows, network protocols are essential for modern connectivity. However, many proprietary protocols used in IoT, ICS/SCADA, and mobile applications lack public documentation, creating “black boxes” that hinder security analysis and open hidden paths for cyber-attacks. Private Protocol Reverse Engineering (PRE) addresses this by inferring protocol specifications from network traffic, binaries, or system behavior without prior knowledge. This survey reviews key PRE methods, including network traffic and program analysis, and highlights how machine learning enhances automation and accuracy. It introduces representative tools and real-world applications in IoT and ICS security and vulnerability discovery. The paper also outlines major challenges such as data dependency, protocol obfuscation, automation limits, and state machine complexity, while proposing future directions like multi-source fusion, deep learning, and domain-specific optimization to strengthen PRE capabilities.

Index Terms—Private protocol reverse engineering; network traffic analysis; program analysis; network security

I. INTRODUCTION

A. research background and significance

In today’s interconnected digital landscape, network protocols are essential for device communication across services such as IoT, ICS/SCADA, and mobile apps [1]. However, many systems use proprietary or undocumented protocols for competitive or practical reasons, creating opaque “black boxes” that hinder third-party understanding and security analysis [2]. This lack of transparency poses serious risks, as critical infrastructure systems often rely on such protocols, potentially harboring undiscovered vulnerabilities.

Private Protocol Reverse Engineering (PRE) addresses these issues by inferring protocol syntax, semantics, and state machines without access to official documentation or source code

This work was supported in part by the National Key Research and Development Plan under Grant 2023YFB3107300, and in part by the National Natural Science Foundation of China (No.62272121, No.U2436208, No.62372129, No.62302115) and Guangdong ST Program under Grant 2024B0101010002, and the Project of Guangdong Key Laboratory of Industrial Control System Security (2024B1212020010), and in part by the Major Key Project of PCL under Grant PCL2024A05, and Guangzhou Science Technology Planning Project under Grant 2024A04J4223, and Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education (No.KLCS20240201), and the Open Project of Key Laboratory Ministry of Industry and Information Technology (No.HK202403347), and Zhejiang Provincial Natural Science Foundation of China under Grants (LTGG24F020008).

[3]. Through the analysis of communication data, binaries, or system behaviour, PRE enables the discovery of hidden communication patterns, vulnerability detection, and system interoperability. By turning “black boxes” into “gray” or even “white” boxes, PRE lays the groundwork for deeper and more effective cybersecurity measures [4].

B. Definition and goal of reverse engineering for proprietary protocols

The core of proprietary protocol reverse engineering (PRE) is to deduce the complete specification of a protocol by systematically analysing communication data flows, protocol handlers, or system runtime behaviour in the absence of a protocol specification or source code [5]. A complete protocol specification usually consists of three key elements: Syntax, Semantics, and State Machine.

- **Syntax:** It refers to the structure and format of data exchange, i.e. the order, length, and data type of each field in a message. For example, a protocol message may contain a fixed field indicating the message type, a field indicating the length of subsequent data, and a variable length data payload [6].
- **Semantics:** The specific meaning of data and control information. Even if you know the format of the field, you need to understand the business logic or instructions it represents. For example, the value of a field might represent “login successful” or “file transfer request” [7].
- **State machine:** It describes the order of interaction and state transitions of legal messages between protocol entities. Communication is usually not an isolated exchange of messages, but follows a set of predefined state and transition rules [8]. For example, a client sends a login request and expects to receive a login response instead of a file transfer request.

The main goal of reverse engineering of private protocols is to achieve a comprehensive inference of these private protocol elements to achieve the following objectives:

- **Understanding unknown communication mechanisms:** Revealing communication rules and data formats hidden behind “black boxes” to lay the foundation for subsequent analysis.

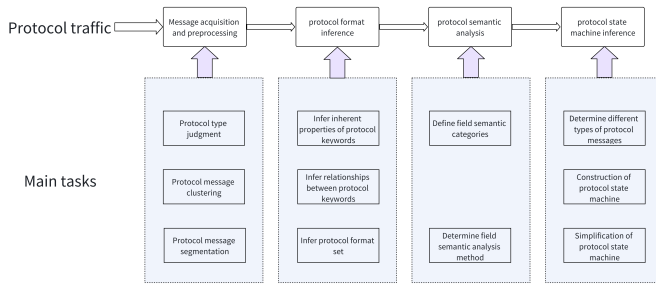


Fig. 1. Architecture of private protocol reverse engineering

- Discovering vulnerabilities in protocol implementations: Analyzing inferred protocol specifications to identify potential security flaws, such as buffer overflows, logic errors, or design flaws, that could be exploited by attackers.
- Assessing security risks for Internet of Things (IoT) and Industrial Control Systems (ICS) devices: Many IoT and ICS devices use proprietary protocols, and reverse engineering of proprietary protocols can help security researchers identify weaknesses in these protocols, assess the device's security posture, and guide security hardening measures.
- Development of compatibility tools or alternative implementations: In the absence of official support or interoperability requirements, compatible clients, servers, or middleware can be developed to enable interoperability between different systems by reverse-engineering inferred protocols [9].

II. REVERSE ANALYSIS OF KEY TECHNOLOGIES AND PROCESSES FOR PROPRIETARY PROTOCOLS

Private protocol reverse analysis is a multi-stage, iterative process, which aims to extract the syntax, semantics and state machine information of the protocol from the original data step by step. Although the specific methods and tools may vary, the core technologies and processes generally follow the following steps: message acquisition and preprocessing, protocol format inference, protocol semantic analysis, and protocol state machine inference [10].

A. Message acquisition and preprocessing

The first step of protocol reverse engineering is to obtain the communication messages of the target protocol [11]. This is typically done through network traffic capture tools such as Wireshark, tcpdump, or through program interception/dynamic staking tools such as Frida. The captured raw data may contain a lot of irrelevant information, so preprocessing is needed to extract messages related to the target protocol. The main tasks of the preconditioning phase include: Traffic filtering and reconstruction: filtering irrelevant protocol data from the original network traffic, and reconstructing the data flow into a complete application layer message session according to known information of the underlying protocol such as TCP/IP (such as IP address, port number and timestamp). For example,

in Wireshark, the "Follow TCP Stream" feature allows you to view complete TCP session data and export it to hexadecimal or ASCII format for later analysis. Message standardization: Standardizes captured network messages into a uniform format, usually including timestamp, source/destination IP address, port number, and original data of the destination protocol. Session grouping: grouping messages into logical sessions based on IP addresses, port numbers, and timestamps of both communicating parties. This grouping is critical for subsequent protocol state machine inference because protocol behavior is session-based. Denoising and Duplication: Eliminate duplicate messages or noisy data to improve the efficiency and accuracy of subsequent analysis [12].

B. protocol format inference

Protocol format inference is a core component of reverse engineering for proprietary protocols and aims to identify the internal structure of protocol messages, including field boundaries, lengths, and data types [13].

1) *Field boundary identification*: Identifying the exact boundaries of the fields in a message is the first task of protocol format inference. Common techniques include:

Statistical analysis: Infer field boundaries by analyzing statistical properties of byte sequences in messages. For example, a frequently occurring fixed byte sequence might be a field delimiter or a message header/trailer. Value distribution statistics and bit flip rate algorithms can help partition protocol fields efficiently because protocol fields typically have a specific value distribution pattern, and variations in bit flip rate may indicate field boundary.

Sequence alignment: By aligning multiple messages, common prefixes, suffixes, fixed patterns, and variation points are identified. NETPLOER [14] uses multiple sequence alignments to align all messages to identify common structure and dynamic parts to infer field boundaries. FieldHunter [15] proposes an "entry distance" metric that recursively merges fields by measuring the distance between adjacent fields to infer field boundaries.

Cluster analysis: similar messages are clustered and then finer field boundary identification is performed within each cluster. Netzob [16] infers protocol structure by clustering messages and partitioning fields.

Dynamic probing: Probing field boundaries by sending modified messages to the server and observing the response. DynPRE [17] analyzes the format of each request through byte-level flip modification and infers field boundaries based on changes in response. For example, modifying a byte may be the beginning or end of a field if it causes a significant change in server response.

2) *Field types and structure inference*: After identifying field boundaries, the next step is to infer the data type (e.g. integer, string, Boolean, timestamp) and structure (e.g. fixed length, variable length, TLV format) of each field.

Range analysis: Analyze the range and distribution of field values, for example, a field may be Boolean if it only appears 0 and 1, or a string if it appears consecutive ASCII characters. In

field type identification, for fields containing invariants, their types correspond to invariant values; for other fields, they are automatically approximated by regular expressions to facilitate verification of data consistency.

Length/offset analysis: identifies whether there is a field in the message that indicates the length or offset of a subsequent field. For example, the value of one field may directly indicate the length of the data block that follows.

Coding format identification: identify the coding mode of the field, such as size endian, ASCII, UTF-8, Base64, Base32, etc. For example, DragonForce ransomware uses Base 32 to encode file names.

Semantic-driven inference: In conjunction with program analysis, infer the type and semantics of a particular field by analyzing how the program handles it. BinaryInferno (BinPRE) [18] uses a semantic-driven approach to infer field types and semantics using atomic semantic detectors and clustering-refinement paradigms. Its performance is superior to existing tools in format extraction and semantic inference.

C. protocol semantic analysis

Protocol semantic analysis aims to understand the actual meaning and functionality of protocol fields and messages, which is often more challenging than syntax analysis because it requires a combination of contextual information and domain knowledge.

Keyword recognition: Identify the "keyword fields" in a message that determine its type or function. NETPLIER [14] uses probabilistic reasoning to identify the most likely keyword fields to correctly cluster network messages. For example, different values of a command word field indicate different operations.

Request-response associations: Infer the semantics of fields by analyzing associations between client requests and server responses. DynPRE [17] explores the different semantics conveyed by different fields by modifying the field content in the request and observing the server's response changes. For example, modifying a User ID field might cause the server to return different user information, revealing the semantics of that field.

Taint analysis: In program analysis, identifying the function and scope of influence of specific data (e.g., user input, network reception) by tracing its propagation and processing through the program. This helps to understand which fields are sensitive data and which are control instructions.

Domain knowledge and heuristics: combine domain-specific knowledge and heuristic rules to aid semantic analysis [19]. For example, in industrial control systems, certain fields may represent equipment status, control commands, or sensor readings.

D. protocol state machine inference

Protocol state machine inference aims to reconstruct the order of interaction and state transitions of legitimate messages between protocol entities [20]. This is critical for understanding the complete behavior of protocols, discovering logical loopholes, and conducting fuzz testing.

Session Sequence Analysis: Messages are grouped into sessions during the message acquisition and preprocessing phases [21]. State machine inference identifies recurring patterns and transitions by analyzing the sequence of messages in each session.

Message clustering and type recognition: grouping messages with the same type or semantics into a class based on format and semantic inference. This is a prerequisite for building a state machine, because state machines are usually transformed based on message type.

Graph construction: Representing different states of a protocol as nodes of a graph and message types or events as edges [22]. By analyzing message sequences, a graph model representing protocol state transitions can be constructed. Netzob [16] is able to infer the protocol's transition graph.

Dynamic probing and state coverage: Actively interact with protocol entities, sending different sequences of messages to explore and cover more protocol states and transitions. AFLNET [23] uses state feedback to guide the fuzzy testing process, thereby exploring the state space of the server more efficiently and dynamically constructing the implemented protocol state machine. DynPRE [17] also utilizes dynamic inference to facilitate downstream state machine inference.

The complexity of protocol state machine inference is that many protocols have a large number of states and complex transition logic, and may have hidden states or depend on specific environmental conditions [24]. Therefore, fully automated inference of accurate and complete protocol state machines remains a challenging research direction.

III. TYPICAL PRIVATE PROTOCOL REVERSE ANALYSIS TOOL

The complexity of reverse analysis of proprietary protocols has led researchers to develop a variety of automated tools to complement or even replace manual analysis. These tools can be divided into network traffic-based tools, program analysis-based tools, and tools that incorporate fuzzy testing, depending on the core methods and analysis objects they adopt.

A. Tools based on network traffic

These tools infer protocol specifications primarily by analyzing captured network packets without accessing program binaries.

- Netzob [16]: An open-source tool for reverse engineering, modeling, traffic generation, and fuzz testing of communication protocols. Its core is a syntax inference engine that supports both passive and active analysis. Netzob handles text-based, delimiter-based, fixed-field, and variable-length protocols. Its reverse engineering process includes message clustering, field partitioning, symbolic abstraction, and state machine inference. It applies bioinformatics and syntactic inference algorithms to bridge academic research and practical applications.
- NETPLIER [14]: A probabilistic protocol reverse-engineering system that analyzes message traces to identify key fields determining message types. It uses proba-

TABLE I
COMPARATIVE ANALYSIS OF TYPICAL PRIVATE PROTOCOL REVERSE ENGINEERING TOOLS

Tool	Core Technologies	Reverse Method	Main Advantages	Performance Metrics	Limitations
Netzob [16]	Statistical analysis, clustering	Network traffic	High precision extraction (5-29% branch coverage)	—	Traffic quality dependent
NETPLIER [14]	Sequence alignment, probabilistic inference	Network traffic	High-precision clustering, strong universality	100% homogeneity, 97.9% completeness	Limited state coverage
DynPRE [17]	Dynamic interaction, message rewriting	Network traffic	Improved field and message type inference	Field: 0.50, V-measure: 0.94	Potential system interference
FieldHunter [25]	Clustering, Entry Distance	Network traffic	Improved field boundary recognition	Superior to existing methods	Focus only on field inference
BinaryInferno [18]	Semantic analysis, atomic detector	Binary analysis	High precision format extraction	Format: 0.73, Semantics: 0.74-0.81	Requires binary access

bilistic reasoning and combines multiple indicators—such as message similarity, structural consistency, and client-server correspondence—to manage uncertainty. By iteratively applying sequence alignment and clustering, NETPLIER performed well in homogeneity and completeness, achieving 100% homogeneity and 97.9% completeness.

- DynPRE [17]: A dynamic protocol reverse-engineering tool that actively interacts with servers to gather richer semantic information. It sends crafted probe messages and analyzes responses through session-specific identifiers and dynamic message rewriting. DynPRE outperforms static analysis tools in field recognition and message type inference, achieving an average perfection score of 0.50 and a V-measure of 0.94. It excels even with limited input data and proprietary protocols.
- FieldHunter [25]: A data-driven tool for reverse engineering power protocols. It uses clustering to infer protocol structure and introduces a novel “entry distance” metric to detect and merge field boundaries. FieldHunter shows superior performance in identifying protocol fields compared to several state-of-the-art methods.

B. Tools based on program analysis

Such tools infer protocol specifications by analyzing the binary files or source code of programs that implement the protocol.

- BinaryInferno (BinPRE) [18]: A semantic-driven binary protocol reverse engineering tool designed to address format inference vulnerabilities and semantic inference limitations faced by prior art binary protocol reverse engineering. BinPRE adopts instruction-based semantic similarity analysis strategy, atomic semantic detector library and clustering-refinement paradigm to improve the adequacy and accuracy of semantic inference. BinPRE outperforms existing tools in format extraction and semantic inference for eight commonly used protocols, with

a format extraction perfection of 0.73 and an F1-score of 0.86. In addition, BinPRE’s field inference results helped improve the effectiveness of protocol fuzz testing, achieving higher branch coverage of 5-29%, and helped discover a new zero-day vulnerability.

- Frida [26]: A powerful dynamic instrumentation toolkit that allows researchers to inspect and manipulate running processes by injecting JavaScript code, supporting multiple platforms such as Linux, Windows, iOS, and Android. In BLE protocol reverse engineering, Frida can hook application API calls, intercept and dump data written to feature values in real time, thus helping to analyze the communication content and behavior of private protocols. Frida’s strength lies in its dynamic nature, bypassing the limitations of static analysis when dealing with obfuscated or dynamic code and directly observing the data flow and function calls of the program at runtime.

C. A tool that incorporates fuzzy testing

Fuzzing is a technique for discovering vulnerabilities by sending a large number of malformed or unexpected inputs to a target system. Some protocol inversion tools combine fuzz testing with protocol inference to improve the efficiency and accuracy of vulnerability discovery [27].

AFLNET [23]: The first grey-box fuzzy testing tool for protocol implementations, addressing the challenges of traditional fuzzy testing in dealing with large server state spaces and the need for well-defined input message sequences [28]. AFLNET adopts mutation method and uses state feedback to guide fuzzy testing process. It is seeded with a corpus of recorded message exchanges between the server and the actual client, requiring no protocol specification or message syntax. AFLNET acts as a client, playing back variations of the original message sequence and identifying server state with the server’s response code, thereby dynamically building and updating the implemented protocol state machine. AFLNET demonstrated significant advantages over existing technologies

in terms of code coverage, state coverage, and vulnerability discovery capabilities in case studies of FTP and RTSP protocols, successfully discovering two critical levels of zero-day vulnerabilities .

IV. APPLICATION SCENARIOS FOR REVERSE ANALYSIS OF PROPRIETARY PROTOCOLS

As a core technology, reverse engineering of proprietary protocols has wide and far-reaching applications in the field of network security, especially in dealing with undocumented or malicious communication protocols.

A. Internet of Things (IoT) Device Security Analysis

The widespread adoption of IoT devices introduces security risks due to their reliance on undocumented proprietary protocols. Reverse engineering these protocols is essential for vulnerability assessment and custom integration.

For instance, research on GoodWe smart meters exposed critical flaws in their encryption implementation, including AES-CBC IV reuse, leading to predictable ciphertext patterns. The devices also retained a Telnet debug interface with default credentials (*admin:admin*) and were vulnerable to factory resets via *nmap* scans. Sensitive data, such as serial numbers, was transmitted in plaintext, while encrypted data remained flawed. These findings spurred the development of tools like *Prometheus Exporters* to monitor and filter traffic.

Another case demonstrated how passive network analysis (*tcpdump/Wireshark*) and MITM techniques (*mitmproxy*) [29] can decode IoT communication patterns. Researchers intercepted HTTPS traffic to study API structures and authentication mechanisms—enabling custom integrations and troubleshooting while adhering to ethical and legal standards.

B. Industrial Control Systems (ICS/SCADA) Safety

Industrial control systems (ICS) and SCADA systems are at the heart of critical infrastructure, and their safety is directly related to national economy and public safety. Many ICS/SCADA devices use proprietary industrial protocols, and reverse engineering of these protocols is critical to identifying system vulnerabilities, developing intrusion detection systems, and ensuring operational security.

PREIUD [30] is a reverse engineering tool for industrial control protocols that integrates unsupervised learning and deep neural networks. It identifies protocol keyword boundaries via a guided voting expert algorithm and classifies formats and semantics using a bidirectional LSTM–CRF model with an attention mechanism. Validated on both proprietary (e.g., S7, FINS) and standard protocols (e.g., Modbus/TCP, Ethernet/IP) in realistic industrial environments, PREIUD achieves 10% higher accuracy in key field recognition than traditional methods and excels in analyzing unknown protocol fields that Wireshark cannot fully parse. This demonstrates the potential of machine learning to handle complex industrial protocols and enhance industrial network security analysis.

C. Vulnerability Mining and Security Protection

Reverse engineering of proprietary protocols is the basis for vulnerability mining and developing advanced security measures. By inferring protocol specifications, security researchers can more effectively design fuzzy test cases, build intrusion detection rules, and develop firewall policies.

For example, AFLNET, as a grey box fuzzy testing tool, can efficiently discover vulnerabilities in protocol implementations by combining protocol inversion and state feedback. It does not require protocol specifications, but rather learns from recorded message exchanges and uses server responses to guide the fuzzy-testing process, thereby exploring the server's state space more efficiently . AFLNET successfully discovered two critical zero-day vulnerabilities in FTP and RTSP protocols, which are difficult to detect by traditional fuzzy testing tools [31]. This indicates that integrating protocol inference capabilities into fuzzy testing processes can significantly improve the depth and efficiency of vulnerability discovery [32].

Furthermore, the inferred protocol specifications can be used to develop more precise firewall rules and intrusion detection system signatures to identify and block malicious communications based on proprietary protocols. By understanding the legitimate behavior of protocols, security systems can better identify anomalous or malicious traffic, thereby improving network defenses.

V. CHALLENGES AND SHORTCOMINGS OF EXISTING RESEARCH

Despite the remarkable progress made in the field of reverse engineering of proprietary protocols, the existing research still faces many challenges and deficiencies, which limit the efficiency, accuracy and universality of reverse engineering of proprietary protocols in practical applications.

A. Data Quality and Sample Dependence

Whether static or dynamic, the accuracy and completeness of inference results based on network traffic analysis are highly dependent on the quality and diversity of captured traffic.

Under-representation of traffic samples: In real-world scenarios, it is difficult to capture a complete traffic sample that covers all protocol features, all message types, and all state transitions. If the traffic sample does not contain a specific message or state, the reverse tool will not be able to infer this part of the protocol information, resulting in incomplete inference results. For example, certain protocol features may only trigger under certain conditions or appear after a long run, making it difficult to capture full traffic.

Interference with low-quality traffic: Network traffic may contain a lot of noise, repeated messages, or non-protocol data, which can interfere with the analysis process and reduce the accuracy of inference [33].

Costly data acquisition: For certain environments (e.g., industrial control networks, embedded devices), capturing high-quality traffic is a challenge in itself and may require specialized hardware or complex network configurations.

B. Confusion and encryption protocol handling

Obfuscation and encryption are common means used by private protocols to protect their specifications, and are one of the biggest challenges facing reverse engineering of private protocols today.

Encryption protocols: Strong encryption algorithms (such as AES, RSA) can effectively hide the syntax and semantic information of the protocol, making it almost impossible to infer the content of the protocol based on traffic analysis alone. While HTTPS traffic can be decrypted through a man-in-the-middle attack (MITM) combined with certificate spoofing, this requires intervention on the target system, and not all encryption protocols use standard TLS/SSL. For custom encryption protocols, additional keys or algorithm reversal are required to decrypt, which often requires deep program analysis capabilities [34]. For example, reverse analysis of GoodWe IoT devices found flaws in their encryption scheme, but their encryption patterns and keys still needed to be identified to fully decrypt them.

Protocol obfuscation: Attackers or developers may employ various obfuscation techniques to hide protocol structure, such as:

Dynamic fields: The length, position, or type of a field changes dynamically at runtime.

Random padding: Random bytes are added to messages to make fixed patterns difficult to identify.

Polymorphism: The same message type appears differently formatted in different contexts or conditions.

Tightly coupled: Protocol fields are tightly coupled to the internal logic of the program, making it difficult to infer their semantics from traffic alone.

These obfuscation techniques make it difficult for traffic analysis methods based on statistics and sequence alignment to effectively identify field boundaries and types. Although tools such as BinaryInferno [18] attempt to identify and de-alias aliased packets by analyzing length variance, frequency patterns, and byte entropy, their effectiveness is limited by the complexity of the aliased packets [35].

C. Efficiency and automation

Despite the proliferation of automated tools, reverse engineering of proprietary protocols remains in many cases a time-consuming and labor-intensive process requiring significant manual intervention and expertise.

Manual analysis is cumbersome: For complex or highly confusing protocols, automated tools may not give satisfactory results, still requiring a lot of manual analysis, debugging and verification by reverse engineers. For example, it took the Samba project 12 years to extract the specification for the SMB protocol, highlighting the complexity and challenge of reverse engineering protocols.

Limitations of tools: Existing tools may have limitations when dealing with specific types of protocols or specific challenges (e.g. encryption, obfuscation, complex state machines). For example, PREIUD's format inference and protocol behavior prediction capabilities are significantly affected when

dealing with proprietary industrial protocols protected by data encryption.

Fragmentation of automated processes: Many tools focus on a specific stage of the protocol reverse process (such as format inference or state machine inference), lacking an end-to-end, highly automated solution that requires manual data conversion and result integration between different tools.

D. Complexity of Protocol State Machine Inference

Protocol state machine inference is one of the most challenging tasks in reverse engineering proprietary protocols because protocol behavior often involves complex logic and a large number of state transitions [36].

State explosion problem: Real-world protocols can have large state spaces, especially when dealing with complex scenarios such as concurrency, timeout, error handling, etc. Exploring all possible protocol states and transition paths is extremely difficult.

Hidden state: Some protocol states may not be directly reflected in messages, but triggered by internal variables or specific events, which makes it difficult to infer from external observations alone.

Non-deterministic behavior: Some protocols may have non-deterministic behavior, i.e., different outputs or state transitions may occur for the same input, which makes state machine inference extremely difficult.

Shortcomings of existing research: Existing research focuses more on the recovery of message types and semantics, while relatively few attempts are made to reconstruct the protocol state machine and often fail to fully capture the dynamic behavior of the protocol. Although tools such as NETPLOER [18] and AFLNET have made some progress in state machine inference, there is still a long way to go before full automation and high-precision inference [37].

VI. FUTURE RESEARCH DIRECTION

In order to meet the challenges faced by the field of reverse engineering of proprietary protocols, future research should focus on the following directions in order to improve the automation, intelligence and universality of reverse engineering of proprietary protocols.

A. Combination of multi-source information and multi-modal analysis

A single data source (such as pure network traffic or pure program binary) is often insufficient to fully infer the specifications of complex protocols. Future research should explore how to efficiently fuse information from different sources for multimodal analysis:

Deep integration of traffic and program analytics: Combine macro behavioral analysis based on network traffic with micro logical analysis based on program analytics. For example, use traffic analysis to quickly identify message types and preliminary formats, and then use program analysis to drill down into the precise semantics and internal processing logic of fields. Dynamic staking tools such as Frida provide strong

support in this regard, bridging traffic and execution-level information.

Combine system logs with memory information: In addition to network traffic and binary files, system logs, memory snapshots, debug information, etc. may also contain rich protocol-related clues. Explore how to automatically extract, correlate, and fuse information from these heterogeneous data sources to build more comprehensive protocol models.

Using domain knowledge and artificial feedback: When it is difficult to fully automate inference, domain expert knowledge and artificial feedback are introduced to form a reverse analysis framework for human-computer cooperation. For example, in the field of industrial control systems, combining the expertise of engineers can significantly improve the accuracy of protocol inference [38].

B. Deep Learning and Reinforcement Learning

Machine learning, especially Deep Learning (DL) and Reinforcement Learning (RL), has demonstrated strong capabilities in dealing with complex pattern recognition and automated decision-making, and is expected to become a core driver of future proprietary protocol reverse engineering research.

More powerful feature representation learning: Using DL models (e.g. Transformer, graph neural networks) to automatically learn more discriminative protocol features from raw byte streams or assembly code overcomes the limitations of traditional feature engineering.

End-to-end protocol inference model: Develop an end-to-end DL model that goes directly from raw data input to protocol specification output, reducing manual intervention in the middle. For example, the sequence-to-sequence (Seq2Seq) model is used to infer protocol formats, or generative adversarial networks (GAN) are used to generate valid protocol messages.

Reinforcement learning driven dynamic probing: RL is applied to dynamic interaction analysis to intelligently generate probe message sequences to maximize protocol state coverage or semantic information acquisition. RL agents can dynamically adjust probing strategies based on server responses to explore the complex state space of the protocol more efficiently [39].

AI approaches to encryption and obfuscation: Study patterns that utilize DL to identify and decrypt encryption protocols, or develop more robust de-obfuscation techniques. For example, adversarial training enables the model to identify confusing features and recover the original protocol structure.

C. Domain-specific protocol optimization

Private protocols in different domains have unique characteristics and challenges, and future research should focus on optimization methods for domain-specific protocols.

Internet of Things (IoT) protocols: IoT devices are resource-constrained and communication protocols can be fragmented and diverse. Research lightweight and efficient reverse engineering methods for proprietary protocols that can adapt to low power consumption and low bandwidth environments and

deal with unique problems in wireless communications (such as packet loss and noise).

Industrial Control System (ICS) Protocol: ICS protocol has high requirements for real-time and reliability, and often involves physical process control. Study private protocol reverse engineering methods that can understand ICS protocol specific semantics (such as control instructions, sensor data) and timing constraints, and consider its application in safety-critical environments [40].

In-vehicle network protocols: With the popularity of smart cars, reverse engineering of proprietary protocols in in-vehicle networks (e.g. CAN, Ethernet) is becoming increasingly important to ensure vehicle safety and interoperability.

Wireless communication protocols: Study reverse engineering for wireless physical layer protocols, considering channel characteristics and signal processing methods unique to wireless communication.

D. Improvement of automation and intelligence level

The ultimate goal is to achieve highly automated and intelligent reverse engineering of proprietary protocols, reducing reliance on human experience.

Automatic test case generation: combining symbolic execution, fuzzy testing and AI technology, automatically generate test cases that cover all functions and states of the protocol, thus improving the efficiency and coverage of dynamic analysis.

End-to-end automation framework: Develop a unified automation platform that integrates all aspects of message acquisition, format inference, semantic analysis, state machine inference, and vulnerability discovery, enabling one-click generation from raw data to protocol specifications and vulnerability reports.

Interpretability AI: When using black box models such as DL/RL, study how to improve the interpretability of their decisions and help reverse engineers understand the basis for inference of the model, so as to better verify and correct the results.

Continuous learning and adaptation: Build proprietary protocol inversion systems that continuously learn and adapt to new protocols or protocol variants, enabling them to continuously update their knowledge base and inference capabilities as the protocol evolves.

VII. CONCLUSION

Private protocol reverse engineering is a critical technique in cybersecurity, essential for IoT security, ICS protection, and vulnerability discovery. This paper reviews its definitions, goals, core methods, key technologies, tools, and applications.

Current methods fall into two main categories: network traffic analysis and program analysis. Traffic-based approaches—such as static analysis (e.g., statistical and clustering methods) and dynamic interaction (e.g., active probing like DynPRE)—infer protocol structures without source code. Program-based methods—like static binary analysis (e.g., BinaryInferno) and dynamic execution (e.g., Frida)—reveal internal logic and data flows. Emerging AI and machine learning

techniques further enhance automation and inference accuracy, aiding tools like AFLNET.

However, challenges remain: dependence on quality traffic samples, difficulty in dealing with encrypted or obfuscated protocols, limited automation, and complex state machine inference. Future directions include integrating multi-source data, leveraging deep/reinforcement learning, optimizing for domain-specific protocols, and enhancing overall automation and intelligence. These advances will strengthen protocol reverse engineering and contribute to a more secure cyberspace.

REFERENCES

- [1] Li Junchen, Cheng Guang, Yang Gangqin. Private Protocol Reverse Engineering Based on Network Traffic: A Survey[J]. *Journal of Computer Research and Development*, 2023, 60(1): 167-190.
- [2] H. Lu, R Guo, C Jin. Automated Anti-obfuscation Technology Based on Capstone and Flow-sensitive Concolic Execution. *Journal of Software*. 2023 Feb 22;34(8):3745-56.
- [3] Huang Qun, Lee P P C, Zhang Zhibin. Exploiting intra-packet dependency for fine-grained protocol format inference[C] //Proc of 2015 IFIP Networking Conf. Piscataway, NJ: IEEE, 2015: 1-9.
- [4] Y. Lv, W. Shi, W. Zhang, H. Lu, and Z. Tian. "Do Not Trust the Clouds Easily: The Insecurity of Content Security Policy Based on Object Storage." *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10462-10470, 2023.
- [5] Yang F, Pan K, Yan C, et al. Systematic Security Analysis of Sensors and Controls in PV Inverters: Threat Validation and Countermeasures[J]. *Sensors*, 2025, 25(5): 1493.
- [6] Jiang S, Zhang Y, Li J, et al. A Survey of Network Protocol Fuzzing: Model, Techniques and Directions[J]. *arXiv preprint arXiv:2402.17394*, 2024.
- [7] H. Lu, C. Jin, X. Helu, X. Du, M. Guizani and Z. Tian. DeepAutoD: Research on Distributed Machine Learning Oriented Scalable Mobile Communication Security Unpacking System. *IEEE Transactions on Network Science and Engineering*. 2022. 9(4): 2052-2065.
- [8] Kleber S, Maile L, Kargl F. Survey of protocol reverse engineering algorithms: Decomposition of tools for static traffic analysis[J]. *IEEE Communications Surveys Tutorials*, 2018, 21(1): 526-561.
- [9] Y. Tan, W. Huang, Y. You, S. Su, H. Lu. Recognizing BGP Communities Based on Graph Neural Network. *IEEE Network*. 2024 Jul 17.
- [10] Jiang Dongxiao, Li Chenggang, Ma Lixin, et al. ABInfer: A novel field boundaries inference approach for protocol reverse engineering[C] //Proc of 2020 IEEE 6th Int Conf on Big Data Security on Cloud (BigDataSecurity). Piscataway, NJ: IEEE, 2020: 19-23.
- [11] Ládi G, Buttyán L, Holczer T. Message format and field semantics inference for binary protocols using recorded network traffic[C] //Proc of 2018 26th Int Conf on Software, Telecommunications and Computer Networks (SoftCOM). Piscataway, NJ: IEEE, 2018: 1-6.
- [12] Atmoko R A, Yang Daoguo. Online monitoring controlling industrial arm robot using MQTT protocol[C] //Proc of 2018 IEEE Int Conf on Robotics, Biomimetics, and Intelligent Computational Systems (Robionetics). Piscataway, NJ: IEEE, 2018: 12-16.
- [13] Kleber S, Kopp H, Kargl F. NEMESYS: Network message syntax reverse engineering by analysis of the intrinsic structure of individual message[C] //Proc of the 12th USENIX Conf on Offensive Technologies. Berkeley, CA: USENIX Association, 2018: 1-8.
- [14] Ye Y, Zhang Z, Wang F, et al. NetPlier: Probabilistic Network Protocol Reverse Engineering from Message Traces[C]//NDSS. 2021.
- [15] Kalleberg K. Frida: Putting the open back into closed software[C]//Open Source Developers Conference. 2015, 62.
- [16] Shim K S, Sohn I, Lee E, et al. Enhance the ICS Network Security Using the Whitelist-based Network Monitoring Through Protocol Analysis[J]. *Journal of Web Engineering*, 2021, 20(1): 1-32.
- [17] Luo Z, Liang K, Zhao Y, et al. DynPRE: Protocol reverse engineering via dynamic inference[C]//Proc. NDSS. 2024: 1-18.
- [18] Jiang J, Zhang X, Wan C, et al. BinPRE: Enhancing Field Inference in Binary Analysis Based Protocol Reverse Engineering[C]//Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024: 3689-3703.
- [19] H. Lu, C. Jin, X. Helu, C. Zhu, N. Guizani, and Z. Tian. "AutoD: Intelligent Blockchain Application Unpacking Based on JNI Layer Deception Call." *IEEE Network*, vol. 35, no. 2, pp. 215-221, 2020.
- [20] Caballero J, Song D. Automatic protocol reverse-engineering: Message format extraction and field semantics inference[J]. *Computer Networks*, 2013, 57(2): 451-474.
- [21] H. Lu, C. Jin, X. Helu, X. Du, M. Guizani, and Z. Tian. "DeepAutoD: Research on Distributed Machine Learning Oriented Scalable Mobile Communication Security Unpacking System." *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 4, pp. 2052-2065, 2021.
- [22] Chandler J, Wick A, Fisher K. BinaryInferno: A Semantic-Driven Approach to Field Inference for Binary Message Formats[C]//NDSS. 2023.
- [23] Aflnet: A greybox fuzzer for network protocols[C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). IEEE, 2020: 460-465.
- [24] Bermudez I, Tongaonkar A, Iliofotou M, et al. Towards automatic protocol field inference[J]. *Computer Communications*, 2016, 84: 40-51.
- [25] R. Jiao, S. Wang, T. Zhang, H. Lu, H. He, and B. B. Gupta. "Adaptive Feature Selection and Construction for Day-Ahead Load Forecasting Using Deep Learning Method." *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4019-4029, 2021.
- [26] Z. Gu, W. Hu, C. Zhang, et al. "Gradient Shielding: Towards Understanding Vulnerability of Deep Neural Networks." *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 921-932, 2020.
- [27] Wang Zhanfeng, Cheng Guang, Ma Weijun, et al. Survey on network protocol reverse engineering approaches based on network trace[J]. *Journal on Software*, 2022, 33(1): 254-273.
- [28] Nategholeslam M, Gray C G, Tomberli B. Implementation of the forward-reverse method for calculating the potential of mean force using a dynamic restraining protocol[J]. *The journal of physical chemistry B*, 2014, 118(49): 14203-14214.
- [29] Xiao Mingming, Luo Yiping. Automatic protocol reverse engineering using grammatical inference[J]. *Intelligent Fuzzy Systems*, 2017, 32(5): 3585-3594.
- [30] Ning B, Zong X, He K, et al. PREIUD: An industrial control protocols reverse engineering tool based on unsupervised learning and deep neural network methods[J]. *Symmetry*, 2023, 15(3): 706.
- [31] Medeiros I, Neves N, Correia M. Detecting and removing Web application vulnerabilities with static analysis and data mining[J]. *IEEE Transactions on Reliability*, 2016, 65(1): 54-69.
- [32] Koo H, Chen Yaohui, Lu Long, et al. Compiler-assisted code randomization[C] //Proc of 2018 IEEE Symp on Security and Privacy (SP). Piscataway, NJ: IEEE, 2018: 461-477.
- [33] You Wei, Liang Bin, Shi Wenchang, et al. TaintMan: An art-compatible dynamic taint analysis framework on unmodified and non-rooted android devices[J]. *IEEE Transactions on Dependable Secure Computing*, 2020, 17(1): 209-222.
- [34] X. Chen, J. Qiu, F. Zhou, H. Lu, L. Zhang, Z. Tian, X. Du, and M. Guizani. "Decentralized Cooperative Caching for Sustainable Metaverse via Meta Self-Supervision Learning." *IEEE Wireless Communications*, vol. 30, no. 5, pp. 96-103, 2023.
- [35] Kuang Boyu, Fu Anmin, Willy S, et al. A survey of remote attestation in Internet of things: Attacks, countermeasures, and prospects[J]. *Computers Security*, 2022, 112:102498.
- [36] Hong Zhen, Yang Chaofeng, Yu Li. R-print: A system residuals-based fingerprinting for attack detection in industrial cyber-physical systems[J]. *IEEE Transactions on Industrial Electronics*, 2021, 68(11): 11458-11469.
- [37] C. Qiao, Y. Zeng, H. Lu, Y. Liu, and Z. Tian. "An Efficient Incentive Mechanism for Federated Learning in Vehicular Networks." *IEEE Network*, vol. 38, no. 5, pp. 189-195, 2023.
- [38] Bursztein E, Cochran G J, Durumeric C Z, et al. Understanding the mirai botnet[C] //Proc of the 26th USENIX Conf on Security Symp. Berkeley, CA: USENIX Association, 2017: 1093-1110.
- [39] Fu Anmin, Zhang Xianglong, Xiong Naixue, et al. VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT[J]. *IEEE Transactions on Industrial Informatics*, 2022, 18(5): 3316-3326.
- [40] Lee S, Song W, Kim K. A study on unknown protocol security[C] //Proc of 2020 International Conf on Information Networking (ICOIN). Piscataway, NJ: IEEE, 2020: 660-662.